

```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.ticker as mticker
from matplotlib.patches import FancyBboxPatch

# --- Data Loading -----

def load_data():
    enc = pd.read_csv("encounters.csv")
    pro = pd.read_csv("procedures.csv")
    pay = pd.read_csv("payers.csv")
    return enc, pro, pay

# --- Prepare -----

def prepare(enc, pro, pay):
    enc = enc.copy()
    # Mixed-timezone safe parsing
    enc["START"] = pd.to_datetime(enc["START"], format="mixed",
    utc=True)
    enc["STOP"] = pd.to_datetime(enc["STOP"], format="mixed",
    utc=True)
    enc["year"] = enc["START"].dt.year
    enc["duration_hrs"] = (enc["STOP"] -
    enc["START"]).dt.total_seconds() / 3600

    pay_map = pay.set_index("Id")["NAME"].to_dict()
    enc["payer_name"] = enc["PAYER"].map(pay_map)
    enc["is_insured"] = (enc["payer_name"] !=
    "NO_INSURANCE").astype(bool)

# --- Admission & 30 days Readmission per year (drop Incomplete final
year) -----

    admissions = enc.groupby("year")["PATIENT"].count()
    enc_s = enc.sort_values(["PATIENT", "START"]).copy()
    enc_s["prev_stop"] = enc_s.groupby("PATIENT")["STOP"].shift(1)
    enc_s["days_gap"] = (enc_s["START"] -
    enc_s["prev_stop"]).dt.total_seconds() / 86400
    readmissions = enc_s[enc_s["days_gap"] <=
    30].groupby("year").size()
    admissions, readmissions = admissions.align(readmissions,
    fill_value=0)
    cutoff = admissions.index.max()
    adm_clean = admissions[admissions.index < cutoff]
    read_clean = readmissions[readmissions.index < cutoff]
    readmit_rate = read_clean.sum() / adm_clean.sum() * 100

# --- Avg LoS by encounter class; inpatient trend by year

```

```

-----
    los_class = enc.groupby("ENCOUNTERCLASS")
["duration_hrs"].mean().sort_values(ascending=False)
    los_yearly = enc[enc["ENCOUNTERCLASS"] ==
"inpatient"].groupby("year")["duration_hrs"].mean()
    los_yearly = los_yearly[los_yearly.index < cutoff]
    overall_los = enc["duration_hrs"].mean()

# --- Mean & Median cost per encounter class
-----
    cost_mean = enc.groupby("ENCOUNTERCLASS")
["TOTAL_CLAIM_COST"].mean().sort_values(ascending = False)
    cost_median = enc.groupby("ENCOUNTERCLASS")
["TOTAL_CLAIM_COST"].median().reindex(cost_mean.index)
    overall_cost = enc["TOTAL_CLAIM_COST"].mean()
    overall_med = enc["TOTAL_CLAIM_COST"].median()
    enc["coverage_pct"] = (enc["PAYER_COVERAGE"] /
enc["TOTAL_CLAIM_COST"].replace(0, float("nan")) * 100
    coverage_by_payer = (enc[enc["payer_name"] !=
"NO_INSURANCE"].groupby("payer_name")
["coverage_pct"].mean().sort_values(ascending = True))

# --- Procedures covered vs uncovered
-----
    pro2 = pro.merge(enc[["Id", "payer_name", "is_insured"]], left_on
= "ENCOUNTER", right_on = "Id", how = "left")
    pro2["is_insured"] = pro2["is_insured"].fillna(False).astype(bool)
    covered = int(pro2["is_insured"].sum())
    uncovered = int((~pro2["is_insured"]).sum())
    payer_counts = pro2["payer_name"].value_counts()

    return (adm_clean, read_clean, readmit_rate, los_class,
los_yearly, overall_los, cost_mean, cost_median,
        overall_cost, overall_med, coverage_by_payer, covered,
uncovered, payer_counts)

# --- KPI Card
-----

def kpi_card(ax, value, label, sub, color):
    ax.set_xlim(0, 1);
    ax.set_ylim(0, 1);
    ax.axis("off")
    ax.add_patch(FancyBboxPatch((0.03, 0.05), 0.94, 0.9, boxstyle =
"round, pad = 0.02", linewidth = 1.2, edgecolor = color, facecolor =
color + "18"))
    ax.text(0.5, 0.72, value, ha = "center", va = "center", fontsize =
20, fontweight = "bold", color = color)
    ax.text(0.5, 0.44, label, ha = "center", va = "center", fontsize =
9, fontweight = "bold", color = "#333333")

```

```

    ax.text(0.5, 0.18, sub, ha = "center", va = "center", fontsize =
8, color = "#666666")

# --- Dashboard
-----

def build_dashboard(enc, pro, pay):
    (adm_clean, read_clean, readmit_rate, los_class, los_yearly,
overall_los, cost_mean, cost_median, overall_cost, overall_med,
coverage_by_payer,
    covered, uncovered, payer_counts) = prepare(enc, pro, pay)
    total_enc = len(enc)
    total_pro = covered + uncovered

    BLUE    = "#378ADD"
    TEAL    = "#1D9E75"
    PURPLE  = "#7F77DD"
    AMBER   = "#EF9F27"
    CORAL   = "#D85A30"
    GRAY    = "#888780"

    fig = plt.figure(figsize = (20, 14), facecolor = "#F8F8F6")
    fig.suptitle("Hospital Patient Records - Dashboard Analysis",
fontsize = 18, fontweight = "bold", y = 0.98, color = "#1a1a1a")

    gs = gridspec.GridSpec(3, 4, figure = fig, hspace = 0.52, wspace =
0.38, left = 0.05, right = 0.97, top = 0.93, bottom = 0.06)

# --- KPI Cards
-----

    kpi_card(fig.add_subplot(gs[0, 0]), f"{total_enc:,}", "Total
Encounters", f"{int(adm_clean.index.min())} -
{int(adm_clean.index.max())}", BLUE)
    kpi_card(fig.add_subplot(gs[0, 1]), f"{readmit_rate:.1f}%", "30 -
Day Readmission Rate", f"{int(read_clean.sum()):,} readmissions",
CORAL)
    kpi_card(fig.add_subplot(gs[0, 2]), f"${overall_cost:,.0f}", "Avg
Cost per Visit", f"Median ${overall_med:,.0f}", AMBER)
    kpi_card(fig.add_subplot(gs[0, 3]), f"{covered / total_pro *
100:.1f}%", "Procedures Covered", f"{covered:,} of {total_pro:,}",
PURPLE)

# --- Admissions and Readmissions
-----

    ax1 = fig.add_subplot(gs[1, 0:2])
    years = list(adm_clean.index)
    w = 0.35
    ax1.bar([y - w / 2 for y in years], adm_clean.values, width = w,
color = BLUE, alpha = 0.88, label = "Admissions")

```

```

    ax1.bar([y + w / 2 for y in years], read_clean.values, width = w,
color = CORAL, alpha = 0.88, label = "Readmissions < 30 days")
    ax1.set_xticks(years)
    ax1.set_xticklabels([str(y) for y in years], fontsize = 9)
    ax1.set_ylabel("Count", fontsize = 8)
    ax1.tick_params(axis="y", labelsize=8)
    ax1.legend(fontsize=8, loc="upper left", framealpha=0.6)
    ax1.spines[["top", "right"]].set_visible(False)
    ax1.set_title("Admissions & 30-Day Readmissions Over Time",
fontweight="bold", fontsize=10)
    peak_yr = int(adm_clean.idxmax())
    ax1.annotate(f"Peak: {peak_yr}\n{int(adm_clean[peak_yr]):,}",
xy=(peak_yr - w / 2, adm_clean[peak_yr]), xytext=(peak_yr - w / 2 -
0.3,
            adm_clean[peak_yr] * 1.05), fontsize=8, color=BLUE,
arrowprops=dict(arrowstyle="->", color=BLUE, lw=0.8))
    for y in years:
        ax1.text(y - w / 2, adm_clean[y] + 15, str(int(adm_clean[y])),
ha="center", fontsize=8, color=BLUE, fontweight="bold")
        ax1.text(y + w / 2, read_clean[y] + 15,
str(int(read_clean[y])), ha="center", fontsize=8, color=CORAL,
fontweight="bold")

```

--- Avg LoS by Encounter Class

```

    ax2 = fig.add_subplot(gs[1, 2:4])
    ec_labels = list(los_class.index)
    ec_vals = list(los_class.values)
    bar_colors2 = [TEAL if v == max(ec_vals) else "#9FE1CB" for v in
ec_vals]
    b2 = ax2.bar(ec_labels, ec_vals, color=bar_colors2,
edgecolor="white", linewidth=0.5, width=0.6)
    ax2.bar_label(b2, fmt="%.2fh", padding=3, fontsize=8,
fontweight="bold")
    ax2.set_ylabel("Avg length of stay (hours)", fontsize=8)
    ax2.tick_params(axis="x", labelsize=8, rotation=12)
    ax2.tick_params(axis="y", labelsize=8)
    ax2.spines[["top", "right"]].set_visible(False)
    ax2.set_title("Avg Length of Stay by Encounter Class",
fontweight="bold", fontsize=10)

    if len(los_yearly) > 1:
        ax_ins = ax2.inset_axes([0.52, 0.42, 0.45, 0.52])
        ax_ins.plot(list(los_yearly.index), list(los_yearly.values),
"o-", color=TEAL, linewidth=1.5, markersize=4)
        ax_ins.fill_between(list(los_yearly.index),
list(los_yearly.values), alpha=0.12, color=TEAL)
        ax_ins.set_title("Inpatient LoS trend", fontsize=7)

```

```

        ax_ins.tick_params(labelsize=6)
        ax_ins.spines[["top", "right"]].set_visible(False)

ax_ins.yaxis.set_major_formatter(mticker.FormatStrFormatter("%.0fh"))

# --- Mean VS Median Cost
-----

    ax3 = fig.add_subplot(gs[2, 0:2])
    classes = list(cost_mean.index)
    x3 = range(len(classes))
    w3 = 0.38
    bm = ax3.bar([i - w3 / 2 for i in x3], cost_mean.values, width=w3,
color=AMBER, label="Mean cost", edgecolor="white")
    bn = ax3.bar([i + w3 / 2 for i in x3], cost_median.values,
width=w3, color="#FAC775", label="Median cost", edgecolor="white")
    ax3.bar_label(bm, fmt="$%.0f", padding=3, fontsize=7,
fontweight="bold", rotation=45)
    ax3.bar_label(bn, fmt="$%.0f", padding=3, fontsize=7,
color="#5F5E5A", rotation=45)
    ax3.set_xticks(list(x3))
    ax3.set_xticklabels(classes, rotation=12, ha="right", fontsize=8)
    ax3.set_ylabel("Cost ($) ", fontsize=8)
    ax3.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, _:
f"${v:,.0f}"))
    ax3.tick_params(axis="y", labelsize=8)
    ax3.legend(fontsize=8, loc="upper right", framealpha=0.6)
    ax3.spines[["top", "right"]].set_visible(False)
    ax3.set_title("Mean vs Median Cost per Visit by Encounter Class",
fontweight="bold", fontsize=10)
    ax3.set_ylim(0, cost_mean.max() * 1.30)

# --- Covered VS Uncovered
-----

    ax4 = fig.add_subplot(gs[2, 2])
    wedges, texts, autotexts = ax4.pie([covered, uncovered],
labels=["Covered\n(insured)", "Uncovered\n(no insurance)"],
autopct="%1.1f%%",
    colors=[PURPLE, GRAY], startangle=90, wedgeprops={"linewidth":
1.2, "edgecolor": "white"}, textprops={"fontsize": 8},
    )
    for at in autotexts:
        at.set_fontsize(8); at.set_color("white");
at.set_fontweight("bold")
    ax4.set_title("Procedures:\nInsured VS Uninsured", fontweight =
"bold", fontsize = 10)

# --- Avg Payer Coverage % bar
-----

```

```

ax5 = fig.add_subplot(gs[2, 3])
p_labels = list(coverage_by_payer.index)
p_values = list(coverage_by_payer.values)
p_colors = [PURPLE if v >= 50 else "#AFA9EC" for v in p_values]
bars5 = ax5.barh(p_labels, p_values, color=p_colors,
edgecolor="white", linewidth=0.5, height=0.65)
ax5.bar_label(bars5, fmt="%.1f%", padding=3, fontsize=7,
fontweight="bold")
ax5.set_xlabel("Avg % of claim cost covered", fontsize=8)
ax5.set_xlim(0, max(p_values) * 1.28 if p_values else 100)
ax5.tick_params(axis="y", labels=7)
ax5.tick_params(axis="x", labels=7)
ax5.spines[["top", "right", "left"]].set_visible(False)
ax5.set_title("Avg Coverage %\nby Insurer", fontweight="bold",
fontsize=10)

return fig

if __name__ == "__main__":
    enc, pro, pay = load_data()
    build_dashboard(enc, pro, pay)

```



